Generics Advanced Types

4B. Generics & Advanced Types

Questions: 8 **Difficulty:** Mid to Senior **Companies:** Google, Amazon, Microsoft, Atlassian, Razorpay, Cred

This is where interviews separate junior from mid-level TypeScript developers. If you can explain generics and utility types from first principles — not just use them — you're ahead of most candidates.

Q 4.9 Explain generics. Why do they exist and how do they work?

Difficulty: Mid **Asked at:** Google, Amazon, Atlassian, Flipkart

Imagine you're building a function that wraps an API response. Without generics, you'd have to either use `any` (losing type safety) or write a separate function for every data type:

TYPESCRIPT Copy

```typescript
// Option 1: any — you lose all type information
function wrapResponse(data: any): { data: any; timestamp: Date } {
  return { data, timestamp: new Date() };
}
const result = wrapResponse({ name: "Ankit" });
result.data.name;  // TypeScript has no idea what .name is — no autocomple
```

Generics solve this. You write the function once, and the type flows through:

TYPESCRIPT Copy

```typescript
function wrapResponse<T>(data: T): { data: T; timestamp: Date } {
  return { data, timestamp: new Date() };
}

// TypeScript infers T from what you pass in
const userResult = wrapResponse({ name: "Ankit", age: 28 });
userResult.data.name;   // TypeScript knows this is string
userResult.data.age;    // TypeScript knows this is number

const postResult = wrapResponse({ title: "Hello", body: "..." });
postResult.data.title;  // TypeScript knows this is string
```

`T` is a type variable — a placeholder that gets filled in when the function is called. You can name it anything, but convention uses `T` for the primary type, `K` for keys, `V` for values, and `E` for elements.

Generic constraints — limiting what T can be:

TYPESCRIPT Copy

```typescript
// Without constraint — T could be anything, even a number
function getProperty<T>(obj: T, key: string) {
  return obj[key];  // Error: T might not have string keys
}

// With constraint — T must be an object with string keys
function getProperty<T extends Record<string, unknown>, K extends keyof T>
  obj: T,
  key: K
): T[K] {
  return obj[key];
}
```

```
getProperty(user, "age");   // Returns number
getProperty(user, "email"); // Error: "email" doesn't exist on this object
```

Generic interfaces and types:

TYPESCRIPT Copy

```
// A generic API response wrapper
interface ApiResponse<T> {
  data: T;
  status: number;
  message: string;
}

// Used with different data types
type UserResponse = ApiResponse<User>;
type PostListResponse = ApiResponse<Post[]>;

// Generic with a default
interface PaginatedResponse<T, M = { page: number; total: number }> {
  items: T[];
  meta: M;
}

// Uses default meta type
const response: PaginatedResponse<User> = {
  items: [{ name: "Ankit" }],
  meta: { page: 1, total: 50 },
};
```

Follow-up: "Can you give a real-world React example?"

TYPESCRIPT Copy

```
// A generic list component
interface ListProps<T> {
  items: T[];
  renderItem: (item: T) => React.ReactNode;
```

```tsx
function List<T>({ items, renderItem, keyExtractor }: ListProps<T>) {
  return (
    <ul>
      {items.map(item => (
        <li key={keyExtractor(item)}>{renderItem(item)}</li>
      ))}
    </ul>
  );
}

// Usage — TypeScript infers T from the items array
<List
  items={users}
  renderItem={(user) => <span>{user.name}</span>}  // user is typed as Use
  keyExtractor={(user) => user.id}
/>
```

Q 4.10 Walk through the built-in utility types. Which ones do you use most and why?

Difficulty: Mid **Asked at:** Nearly every company — Google, Amazon, Flipkart, Razorpay, Swiggy, Microsoft

TypeScript ships with utility types that transform existing types. You don't need to memorize all of them, but interviewers expect you to know the ones you'd use daily.

`Partial<T>` **— makes all properties optional**

TYPESCRIPT Copy

```tsx
interface User {
  name: string;
  email: string;
  age: number;
}
```

```
}

updateUser("1", { name: "Ankit" });  // Valid - only updating name
```

`Required<T>` — makes all properties required (opposite of Partial)

TYPESCRIPT Copy

```
interface Config {
  host?: string;
  port?: number;
  debug?: boolean;
}

// After merging with defaults, all properties are guaranteed to exist
const finalConfig: Required<Config> = {
  host: "localhost",
  port: 3000,
  debug: false,
};
```

`Pick<T, K>` — selects specific properties

TYPESCRIPT Copy

```
// Only take what you need for the profile card
type ProfileCard = Pick<User, "name" | "avatar" | "role">;
```

`Omit<T, K>` — removes specific properties

TYPESCRIPT Copy

```
// Everything from User except the password
type PublicUser = Omit<User, "password" | "ssn">;

// Common in forms - omit auto-generated fields
type CreateUserInput = Omit<User, "id" | "createdAt" | "updatedAt">;
```

TYPESCRIPT Copy

```typescript
// A map of status codes to messages
type StatusMessages = Record<number, string>;

const messages: StatusMessages = {
  200: "OK",
  404: "Not Found",
  500: "Server Error",
};

// A map of user roles to permissions
type RolePermissions = Record<"admin" | "editor" | "viewer", string[]>;
```

`Readonly<T>` — makes all properties read-only

TYPESCRIPT Copy

```typescript
interface State {
  count: number;
  items: string[];
}

const state: Readonly<State> = { count: 0, items: [] };
state.count = 1;  // Error: Cannot assign to 'count' because it is a read-
```

`ReturnType<T>` — extracts the return type of a function

TYPESCRIPT Copy

```typescript
function fetchUser() {
  return { id: "1", name: "Ankit", role: "admin" as const };
}

type UserData = ReturnType<typeof fetchUser>;
// { id: string; name: string; role: "admin" }
```

TYPESCRIPT Copy

```typescript
function createOrder(items: CartItem[], customer: Customer, rush: boolean)

type OrderParams = Parameters<typeof createOrder>;
// [CartItem[], Customer, boolean]
```

`Exclude<T, U>` and `Extract<T, U>` — filter union types

TYPESCRIPT Copy

```typescript
type Status = "loading" | "success" | "error" | "idle";

type ActiveStatus = Exclude<Status, "idle">;  // "loading" | "success" | "
type FailureStatus = Extract<Status, "error">; // "error"
```

`NonNullable<T>` — removes null and undefined

TYPESCRIPT Copy

```typescript
type MaybeUser = User | null | undefined;
type DefiniteUser = NonNullable<MaybeUser>;  // User
```

The ones you'll use daily: `Partial`, `Pick`, `Omit`, `Record`, and `ReturnType`. These come up in React props, API types, form handling, and state management constantly.

Q 4.11 What are mapped types? How do you create new types from existing ones?

Difficulty: Senior **Asked at:** Google, Microsoft, Razorpay

Mapped types let you transform every property in a type systematically. Think of them as a `map()` function but for types instead of arrays.

TYPESCRIPT Copy

```typescript
// This is how Readonly<T> works under the hood
type MyReadonly<T> = {
  readonly [K in keyof T]: T[K];
};

// This is how Partial<T> works under the hood
type MyPartial<T> = {
  [K in keyof T]?: T[K];
};

// Make all properties nullable
type Nullable<T> = {
  [K in keyof T]: T[K] | null;
};

interface User {
  name: string;
  age: number;
  email: string;
}

type NullableUser = Nullable<User>;
// { name: string | null; age: number | null; email: string | null }
```

Key remapping with `as` — changing property names:

TYPESCRIPT Copy

```typescript
// Add "get" prefix to every property and make them functions
type Getters<T> = {
  [K in keyof T as `get${Capitalize<string & K>}`]: () => T[K];
};

type UserGetters = Getters<User>;
// { getName: () => string; getAge: () => number; getEmail: () => string }

// Remove specific properties during mapping
type RemoveKind<T> = {
```

Practical example — form field states:

TYPESCRIPT Copy

```typescript
interface LoginForm {
  email: string;
  password: string;
}

// Generate validation state for every field
type FormErrors<T> = {
  [K in keyof T]?: string;  // Error message or undefined
};

type FormTouched<T> = {
  [K in keyof T]: boolean;
};

type LoginErrors = FormErrors<LoginForm>;
// { email?: string; password?: string }

type LoginTouched = FormTouched<LoginForm>;
// { email: boolean; password: boolean }
```

When you change `LoginForm` — say you add a `rememberMe` field — the error and touched types update automatically. You don't maintain them separately.

Q 4.12 Explain conditional types and the `infer` keyword.

Difficulty: Senior **Asked at:** Google, Microsoft, Amazon

Conditional types are TypeScript's version of a ternary — they choose between two types based on a condition:

TYPESCRIPT Copy

```typescript
type IsString<T> = T extends string ? true : false;

type A = IsString<"hello">;  // true
type B = IsString<42>;       // false
type C = IsString<string>;   // true
```

Practical example — unwrapping promises:

TYPESCRIPT Copy

```typescript
// If T is a Promise, extract the inner type. Otherwise, return T as-is.
type Unwrap<T> = T extends Promise<infer U> ? U : T;

type A = Unwrap<Promise<string>>;  // string
type B = Unwrap<Promise<number>>;  // number
type C = Unwrap<string>;           // string (not a promise, returned as-i
```

The `infer` keyword is like a type-level variable declaration. It says "I don't know what this type is yet — figure it out from the structure."

`infer` for extracting function return types:

TYPESCRIPT Copy

```typescript
// This is how ReturnType<T> works under the hood
type MyReturnType<T> = T extends (...args: any[]) => infer R ? R : never;

type A = MyReturnType<() => string>;           // string
type B = MyReturnType<(x: number) => boolean>; // boolean
```

`infer` for extracting array element types:

TYPESCRIPT Copy

```typescript
type ElementType<T> = T extends (infer E)[] ? E : T;
```

`infer` for extracting component props:

TYPESCRIPT Copy

```typescript
type PropsOf<T> = T extends React.ComponentType<infer P> ? P : never;

type ButtonProps = PropsOf<typeof Button>;
// Extracts the props type from any React component
```

Distributive conditional types — applying the condition to each member of a union:

TYPESCRIPT Copy

```typescript
type ToArray<T> = T extends unknown ? T[] : never;

type A = ToArray<string | number>;
// Distributes: (string extends unknown ? string[] : never) | (number exte
// Result: string[] | number[]
// NOT: (string | number)[]
```

This distribution is why `Exclude<T, U>` works:

TYPESCRIPT Copy

```typescript
// Exclude distributes over the union
type Exclude<T, U> = T extends U ? never : T;

type Result = Exclude<"a" | "b" | "c", "a">;
// "a" extends "a" ? never : "a"  → never
// "b" extends "a" ? never : "b"  → "b"
// "c" extends "a" ? never : "c"  → "c"
// Result: "b" | "c"
```

Follow-up: "When do you actually use conditional types in real code?"

Most developers consume conditional types through utility types rather than writing their own. But when building shared libraries, design systems, or complex generic components, conditional types let you create type-safe APIs that adapt based on what the user passes in:

TYPESCRIPT Copy

```typescript
// Component that renders differently based on the 'as' prop
type PolymorphicProps<E extends React.ElementType> = {
  as?: E;
} & Omit<React.ComponentPropsWithoutRef<E>, "as">;

// Now <Button as="a" href="/home" /> is type-safe
// and <Button as="a" onClick={...} /> also works
// but <Button as="a" href={123} /> is a type error
```

Q 4.13 What are template literal types?

Difficulty: Senior **Asked at:** Google, Razorpay, Microsoft

Template literal types let you manipulate strings at the type level, using the same backtick syntax as JavaScript template literals:

TYPESCRIPT Copy

```typescript
type Greeting = `Hello, ${string}`;

const a: Greeting = "Hello, Ankit";    // Valid
const b: Greeting = "Hello, World";    // Valid
const c: Greeting = "Hi there";        // Error: doesn't match pattern
```

Building event handler types:

TYPESCRIPT Copy

```typescript
type EventName = "click" | "focus" | "blur";
type Handler = `on${Capitalize<EventName>}`;
```

CSS unit types:

TYPESCRIPT Copy

```typescript
type CSSUnit = "px" | "rem" | "em" | "%";
type CSSValue = `${number}${CSSUnit}`;

const width: CSSValue = "100px";   // Valid
const height: CSSValue = "2rem";   // Valid
const margin: CSSValue = "auto";   // Error
```

Typed API routes:

TYPESCRIPT Copy

```typescript
type ApiRoute = `/api/${string}`;
type UserRoute = `/api/users/${number}`;

function fetchData(url: ApiRoute) { ... }

fetchData("/api/users");       // Valid
fetchData("/api/posts/123");   // Valid
fetchData("/users");           // Error: must start with /api/
```

With mapped types — creating getter/setter pairs:

TYPESCRIPT Copy

```typescript
type Getters<T> = {
  [K in keyof T as `get${Capitalize<string & K>}`]: () => T[K];
};

type Setters<T> = {
  [K in keyof T as `set${Capitalize<string & K>}`]: (value: T[K]) => void;
};

interface Config {
  host: string;
```

```
type ConfigAccessors = Getters<Config> & Setters<Config>;
// {
//   getHost: () => string;
//   getPort: () => number;
//   setHost: (value: string) => void;
//   setPort: (value: number) => void;
// }
```

TypeScript provides built-in string manipulation types: `Uppercase`, `Lowercase`, `Capitalize`, and `Uncapitalize` that work with template literal types.

Q 4.14 Explain `keyof` and indexed access types.

Difficulty: Mid **Asked at:** Amazon, Flipkart, Atlassian

`keyof` **— creates a union of all property names:**

TYPESCRIPT Copy

```
interface User {
  id: number;
  name: string;
  email: string;
}

type UserKeys = keyof User;  // "id" | "name" | "email"
```

This is the foundation for type-safe property access:

TYPESCRIPT Copy

```
function getValue<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}

const user: User = { id: 1, name: "Ankit", email: "a@b.com" };
```

```
getValue(user, "phone");  // Error: "phone" is not in keyof User
```

Indexed access types — look up the type of a specific property:

TYPESCRIPT Copy

```
type UserName = User["name"];        // string
type UserId = User["id"];            // number
type NameOrEmail = User["name" | "email"];  // string (both are string)
```

Combined with arrays:

TYPESCRIPT Copy

```
const statuses = ["loading", "success", "error"] as const;
type Status = typeof statuses[number];  // "loading" | "success" | "error"
```

`[number]` is the indexed access — it says "give me the type at any numeric index of this array," which is the union of all element types.

Practical example — typed event system:

TYPESCRIPT Copy

```
interface EventMap {
  login: { userId: string; timestamp: Date };
  logout: { userId: string };
  purchase: { itemId: string; amount: number };
}

function emit<K extends keyof EventMap>(event: K, payload: EventMap[K]) {
  // ...
}

emit("login", { userId: "123", timestamp: new Date() });  // Valid
```

Q 4.15 When and why would you use function overloads?

Difficulty: Mid **Asked at:** Google, Microsoft, Amazon

Function overloads let you describe multiple ways to call the same function, each with different parameter and return types:

TYPESCRIPT Copy

```typescript
// Overload signatures — describe what's possible
function createElement(tag: "a"): HTMLAnchorElement;
function createElement(tag: "div"): HTMLDivElement;
function createElement(tag: "input"): HTMLInputElement;
function createElement(tag: string): HTMLElement;

// Implementation signature — handles all cases
function createElement(tag: string): HTMLElement {
  return document.createElement(tag);
}

const anchor = createElement("a");     // TypeScript returns HTMLAnchorEle
const div = createElement("div");       // TypeScript returns HTMLDivEleme
const custom = createElement("my-el");  // TypeScript returns HTMLElement
```

Without overloads, the return type would always be `HTMLElement`, and you'd lose the specific element types.

When to use overloads:

When the return type depends on the input type, and a single generic signature can't express that relationship cleanly:

TYPESCRIPT Copy

```
function parse(input: string, format: "csv"): string[][];
function parse(input: string, format: "json" | "csv"): object | string[][]
  if (format === "json") return JSON.parse(input);
  return input.split("
").map(row => row.split(","));
}

const data = parse(content, "json");  // TypeScript knows this is object
const rows = parse(content, "csv");   // TypeScript knows this is string[]
```

When NOT to use overloads: When a union type or generic handles it fine. Overloads add complexity. If you can express the relationship with `T extends X ? Y : Z`, that's usually cleaner.

Q 4.16 What is declaration merging and module augmentation? When do you need them?

Difficulty: Senior **Asked at:** Google, Microsoft, Razorpay

Declaration merging is when TypeScript combines multiple declarations with the same name into a single definition. This only works with `interface`, not `type`:

TYPESCRIPT Copy

```
// First declaration
interface Window {
  title: string;
}

// Second declaration — merged with the first
interface Window {
  analytics: {
    track: (event: string) => void;
  };
}
```

```
window.analytics.track("page_view");
```

Why this exists: It lets you extend third-party library types without modifying their source code. This is how DefinitelyTyped works — `@types/express` extends Node.js's built-in `Request` type by declaring additional interfaces.

Module augmentation — extending types from other modules:

TYPESCRIPT Copy

```typescript
// Extend Express's Request type to include a custom user property
declare module "express" {
  interface Request {
    user?: {
      id: string;
      role: string;
    };
  }
}

// Now req.user is typed in your route handlers
app.get("/profile", (req, res) => {
  const userId = req.user?.id;  // TypeScript knows about this
});
```

Augmenting environment variables:

TYPESCRIPT Copy

```typescript
// env.d.ts
declare namespace NodeJS {
  interface ProcessEnv {
    DATABASE_URL: string;
    API_KEY: string;
    NODE_ENV: "development" | "production" | "test";
  }
}
```

Augmenting CSS modules:

TYPESCRIPT Copy

```typescript
// For typed CSS module imports
declare module "*.module.css" {
  const classes: Record<string, string>;
  export default classes;
}

declare module "*.module.scss" {
  const classes: Record<string, string>;
  export default classes;
}
```

When you'll actually use this:

- Adding types for `window` globals (analytics, feature flags, env vars)
- Extending Express/Fastify request objects with authentication data
- Adding types for CSS/SVG/image imports in bundlers
- Augmenting third-party libraries with custom methods or properties

Important: Module augmentation only works inside a module (a file with at least one `import` or `export` statement). If your file has no imports/exports, TypeScript treats it as a script and the augmentation becomes a global declaration, which might not be what you want.

QUICK CHECK — no marks, just for you.

1. You are typing an updateUser(id, updates) function where the caller may send any subset of a User's fields — sometimes just the name, sometimes just the age, sometimes everything. Which built-in utility type correctly describes the `updates` parameter?

- ◯ Required<User>

- Readonly<User>
- Partial<User>

Check answer

2. Given `function getProperty<T extends Record<string, unknown>, K extends keyof T>(obj: T, key: K): T[K]` and `const user = { name: "Ankit", age: 28 }`, why does `getProperty(user, "email")` fail to compile?

- T extends Record<string, unknown> forbids objects that contain number values like age
- K is constrained to keyof T, and "email" is not a key of the inferred type of user
- The return type T[K] cannot be resolved unless the key is passed as a variable rather than a literal
- Generic constraints are erased at compile time, so any missing key is only caught at runtime

Check answer

3. With `type ToArray<T> = T extends unknown ? T[] : never`, what does `ToArray<string | number>` resolve to, and why?

- unknown[], because T widens to unknown before the array type is applied
- string[] | number[], because the conditional type distributes over each member of the union
- never, because a union is not assignable to unknown
- (string | number)[], because the whole union is checked against unknown as a single type

Previous
Typescript Fundamentals

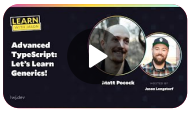
Next
Typescript In Practice

LEARN MORE

ARTICLES & DOCS

Generics (TS Handbook)
typescriptlang.org

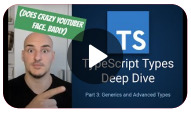
VIDEOS



Let's learn generics (Matt Pocock)
YouTube



Advanced TypeScript: utility types, generics...
YouTube



Generics & advanced types deep dive
YouTube

Hand-picked references for this lesson.